

Estudo e revisão: primeira API backend com Express e MySQL – Parte 1

Este material revisa o conteúdo inicial do roteiro da primeira API CRUD com JavaScript, Express e MySQL, considerando o conteúdo até a criação das primeiras rotas no servidor.

O foco aqui é revisar a parte de backend com Node.js, estrutura do projeto, dependências, configuração, variáveis de ambiente, conexão com banco e início do servidor Express.

Observação: este arquivo não revisa o conteúdo de SQL.

1. Objetivo do projeto

O projeto tem como objetivo criar uma API backend simples usando:

- JavaScript
- Node.js
- Express
- MySQL

A API será responsável por um CRUD de produtos.

CRUD significa:

- **Create** : criar registros
- **Read** : consultar registros
- **Update** : atualizar registros
- **Delete** : remover registros

Neste projeto, as operações serão feitas por meio de rotas HTTP.

2. O que é uma API backend?

Uma API backend é uma aplicação que recebe requisições, processa regras e responde com dados.

Neste caso, o backend:

- Recebe requisições HTTP

- Acessa o banco de dados
- Executa operações relacionadas aos produtos
- Retorna respostas em JSON

Exemplo de resposta em JSON:

```
{  
  "message": "API CRUD com Express e MySQL"  
}
```

Este projeto não possui frontend. Ou seja, não há páginas HTML, telas ou interface visual. Os testes são feitos com ferramentas como Postman, Insomnia ou curl.

3. Pré-requisitos

Para acompanhar o projeto, é necessário ter:

- `Node.js` : ambiente que permite executar JavaScript fora do navegador
- `npm` : gerenciador de pacotes do Node.js
- `MySQL Server` : banco de dados usado pela API
- `Postman` , `Insomnia` ou `curl` : ferramentas para testar requisições
- Um editor de código, como VS Code ou Cursor

4. Criação do projeto Node.js

O projeto começa com a criação de uma pasta:

```
mkdir primeira-api-crud  
cd primeira-api-crud
```

Depois, é criado o arquivo `package.json` :

```
npm init -y
```

O `package.json` guarda informações importantes do projeto, como:

- Nome do projeto
- Versão
- Scripts de execução

- Dependências instaladas

5. Dependências do projeto

As principais dependências instaladas são:

```
npm install express mysql2 dotenv
```

Também é instalado o `nodemon` como dependência de desenvolvimento:

```
npm install -D nodemon
```

Para que serve cada pacote?

`express`

Cria o servidor HTTP e permite definir rotas da API.

`mysql2`

Permite que a aplicação Node.js se conecte ao MySQL.

`dotenv`

Carrega variáveis de ambiente a partir de um arquivo `.env`.

`nodemon`

Reinicia o servidor automaticamente sempre que algum arquivo é alterado durante o desenvolvimento.

6. Scripts do `package.json`

No `package.json`, os scripts ajudam a executar o projeto com comandos mais simples.

Exemplo:

```
{
  "scripts": {
    "dev": "nodemon src/server.js",
    "start": "node src/server.js"
  }
}
```

```
}  
}
```

O script `dev` é usado durante o desenvolvimento:

```
npm run dev
```

O script `start` é usado para executar o projeto com Node.js diretamente:

```
npm start
```

7. Estrutura inicial de arquivos

A estrutura inicial do projeto é:

```
primeira-api-crud/  
├── src/  
│   ├── database.js  
│   └── server.js  
├── .env  
├── .gitignore  
├── package.json  
└── package-lock.json
```

Papel de cada arquivo

`src/`

Pasta onde fica o código-fonte da aplicação.

`src/database.js`

Arquivo responsável por configurar e exportar a conexão com o banco de dados.

`src/server.js`

Arquivo principal do servidor Express. Nele ficam a criação da aplicação, os middlewares e as rotas iniciais.

`.env`

Arquivo usado para guardar configurações sensíveis ou variáveis, como porta da aplicação e dados de conexão.

`.gitignore`

Arquivo usado para informar ao Git quais arquivos ou pastas não devem ser versionados.

`package.json`

Arquivo de configuração do projeto Node.js.

`package-lock.json`

Arquivo gerado automaticamente para registrar as versões exatas das dependências instaladas.

8. Variáveis de ambiente

O arquivo `.env` guarda configurações que podem mudar de um ambiente para outro.

Exemplo:

```
PORT=3000
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=sua_senha
DB_NAME=primeira_api
```

Essas informações são acessadas no código por meio de `process.env`.

Exemplo:

```
const port = process.env.PORT || 3000;
```

Neste exemplo, a aplicação usa a porta definida em `PORT`. Se ela não existir, usa `3000` como valor padrão.

9. Importância do `.gitignore`

O `.gitignore` evita que arquivos desnecessários ou sensíveis sejam enviados para o repositório.

Exemplo:

```
node_modules
.env
```

Por que ignorar `node_modules`?

A pasta `node_modules` pode ser muito grande e é recriada automaticamente com:

```
npm install
```

Por que ignorar `.env`?

O `.env` pode conter senhas e configurações privadas, como usuário e senha do banco.

10. Conexão com o banco no `database.js`

O arquivo `src/database.js` cria e exporta a conexão com o banco:

```
const mysql = require('mysql2/promise');
require('dotenv').config();
```

```
const pool = mysql.createPool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0,
});
```

```
module.exports = pool;
```

Pontos importantes

```
require('mysql2/promise')
```

Importa a versão com suporte a Promises do pacote `mysql2`, permitindo usar `async` e `await`.

```
require('dotenv').config()
```

Carrega as variáveis definidas no arquivo `.env`.

```
mysql.createPool
```

Cria um pool de conexões. Isso permite reaproveitar conexões com o banco, em vez de abrir uma nova conexão para cada requisição.

```
module.exports = pool
```

Exporta o pool para que ele possa ser usado em outros arquivos, como `server.js`.

11. Criação do servidor Express

No arquivo `src/server.js`, o servidor começa assim:

```
const express = require('express');
require('dotenv').config();

const db = require('./database');

const app = express();
const port = process.env.PORT || 3000;

app.use(express.json());
```

O que cada parte faz?

```
const express = require('express')
```

Importa o Express.

```
const db = require('./database')
```

Importa a conexão configurada no arquivo `database.js`.

```
const app = express()
```

Cria uma aplicação Express.

```
const port = process.env.PORT || 3000
```

Define a porta em que o servidor será executado.

```
app.use(express.json())
```

Permite que a API receba dados no formato JSON no corpo das requisições.

12. Primeiras rotas

A primeira rota serve para testar se a API está online:

```
app.get('/', (req, res) => {  
  res.json({ message: 'API CRUD com Express e MySQL' });  
});
```

Essa rota responde a uma requisição **GET** na raiz da API.

Exemplo de endereço:

```
http://localhost:3000
```

A rota de listagem de produtos começa assim:

```
app.get('/produtos', async (req, res) => {  
  try {  
    const [produtos] = await db.query('...');  
    res.json(produtos);  
  } catch (error) {  
    res.status(500).json({ message: 'Erro ao listar produtos' });  
  }  
});
```

Pontos importantes dessa rota

```
app.get('/produtos', ...)
```

Define uma rota HTTP do tipo **GET**.

```
async
```

Indica que a função pode usar **await** para esperar operações assíncronas.

```
await db.query(...)
```

Executa uma consulta no banco e espera o resultado.

`try/catch`

Ajuda a tratar erros. Se algo falhar, a API responde com status `500`.

`res.json(produtos)`

Envia a resposta em formato JSON.

13. Resumo para memorização

- O `Node.js` permite executar JavaScript no backend.
- O `npm` gerencia pacotes e scripts do projeto.
- O `Express` cria o servidor e as rotas.
- O `mysql2` conecta a aplicação ao MySQL.
- O `dotenv` carrega variáveis do arquivo `.env`.
- O `nodemon` reinicia o servidor durante o desenvolvimento.
- O `src/server.js` é o ponto principal da API.
- O `src/database.js` concentra a configuração de conexão.
- O `.env` guarda configurações sensíveis ou variáveis.
- O `.gitignore` evita versionar arquivos desnecessários ou privados.
- `express.json()` permite receber JSON no corpo da requisição.
- `async/await` facilita trabalhar com operações assíncronas.
- `try/catch` permite tratar erros nas rotas.

14. Checklist de revisão

Antes de avançar, confira se você consegue responder:

- O que é uma API backend?
- O que significa CRUD?
- Para que serve o `package.json`?
- Qual é a diferença entre dependência normal e dependência de desenvolvimento?
- Para que serve o `nodemon`?
- Por que usamos `.env`?
- Por que o `.env` deve ficar no `.gitignore`?
- Para que serve o arquivo `database.js`?
- Para que serve o arquivo `server.js`?
- O que faz `app.use(express.json())`?
- O que é uma rota `GET`?

- Para que usamos `try/catch` em rotas assíncronas?